RESEARCH DESIGN DOCUMENT

ViewTree

Resource-Aware On-Device Spatial Reasoning via Confidence-Guided Viewpoint Branching and Fusion

Purpose: Detailed motivation, method specification, training plan, and experimental roadmap

Status: Working design; empirical results and final implementation choices remain to be validated

Target: Mobile/embodied systems paper with reasoning and systems co-design contributions

Date: 24 August 2026

> **Central claim:** A lightweight VLM should not receive every reconstructed view or follow only one camera trajectory. It should learn when additional viewpoints are worth acquiring, preserve competing hypotheses while exploring, retain branches according to calibrated VLM confidence, and fuse only the evidence that improves the final spatial decision under the device budget.

Executive summary

ViewTree converts recent egocentric observations into an explicit 3D world memory and treats that memory as an interactive reasoning substrate. A fine-tuned VLM controls a virtual camera, receives encoded renderings of requested viewpoints, and can continue a trajectory, branch into alternatives, fuse retained evidence, or stop. The central reasoning contribution is not reconstruction: it is confidence-guided conditional computation over physically grounded viewpoints.

At each branch point, candidate trajectories inherit a shared reasoning prefix but continue independently. A learned VLM confidence head estimates the probability that each partial branch will eventually produce the correct answer. Given a device-determined branch budget, the system keeps the highest-confidence branches; geometry is used only to reject invalid renderings or exact duplicates. High-confidence disagreement triggers additional evidence acquisition rather than premature pruning. Selected views are finally presented with their camera poses for joint reasoning.

The systems contribution is a mobile runtime that shares pre-branch VLM state, batches sibling rendering and visual encoding, applies progressive rendering fidelity, caches pose-indexed observations, and adapts branch width and depth to latency, energy, memory, and thermal constraints. The

experiment plan therefore evaluates both reasoning quality and the complete accuracy–resource Pareto frontier on real mobile/embodied hardware.

Document map

1. Sections 1–2 establish the motivation, closest-work boundary, research questions, scope, and assumptions.
2. Sections 3–5 define the architecture, formal problem, inference algorithm, and confidence-guided branch lifecycle.
3. Section 6 specifies the staged training procedure and how branch confidence receives outcome-based supervision.
4. Section 7 describes the mobile runtime and its resource-control interface.
5. Sections 8–10 provide the experiment matrix, statistical protocol, implementation roadmap, risks, and paper positioning.

1. Motivation and research opportunity

1.1 From visual perception to active spatial reasoning

Mobile and embodied devices increasingly use lightweight VLMs to interpret camera observations locally. This offers low communication latency and stronger privacy, but it also exposes a capability gap: spatial questions frequently depend on evidence that is absent from the current image, distributed across historical viewpoints, or hidden by occlusion. Recognizing the objects is not sufficient; the system must decide where to look, acquire the missing evidence, and combine observations expressed in different camera coordinate systems.

Examples include determining whether two objects lie on opposite sides of a barrier, identifying which doorway becomes visible after a turn, estimating whether an object is reachable from another side of a table, and deciding which path remains traversable behind an occluder. These questions require active viewpoint reasoning rather than passive frame understanding.

1.2 Why generative imagination is not enough

One approach lets a VLM instruct a generative world model to synthesize unseen viewpoints. This can provide flexible visual imagination, but large camera transformations are difficult to keep geometrically consistent with the original observations. Generation also adds a substantial model invocation to every reasoning step. On a mobile platform, repeated world-model calls compete with the vision encoder and

VLM decoder for memory, energy, and thermal headroom. AVIC shows why imagination must be treated as an adaptive resource rather than invoked unconditionally [5].

1.3 Explicit 3D memory solves grounding, not reasoning

A geometry foundation model such as VGGT can instead infer camera parameters, depth, point maps, and 3D tracks from multiple views [2]. This explicit memory grounds novel viewpoints in reconstructed geometry and avoids asking a generative model to invent the complete scene. Nevertheless, directly exposing the entire point cloud or a large set of renderings to a small VLM creates a different bottleneck: most geometry is irrelevant to the query, visual tokens become redundant, and the model still lacks a policy for deciding which view should be inspected next.

1.4 Single-chain exploration is brittle

Think3D and Chain-of-View demonstrate that iterative virtual-camera manipulation can improve spatial reasoning [3, 4]. Their basic reasoning loop, however, is dominated by a single sequence of actions. A mistaken early camera decision may lead to uninformative regions, and one trajectory may not contain evidence distributed across opposing directions. Static multi-view selection, represented by cdViews [6], can retain critical and diverse renderings, but it does not perform online, branch-conditional evidence acquisition. Generic multimodal tree search, represented by VisuoThink [7], explores multiple reasoning paths but does not specialize the search policy and systems runtime for camera trajectories over an explicit mobile 3D memory.

1.5 The unresolved challenge

> **Research question:** How can a small on-device VLM decide when to fork spatial exploration, which partial camera trajectories remain promising, when complementary branches should be combined, and how deeply reasoning should continue under changing device constraints?

Unconditional branching is not the answer. It multiplies rendering, encoding, and decoding costs; redundant viewpoints can distract the VLM; and low-quality renderings from sparse regions can increase confidence without increasing correctness. The system must therefore learn a conditional branch structure from task outcomes, while the runtime enforces the number of branches the current device can afford.

1.6 Closest-work boundary

Work	Direct overlap	Boundary for ViewTree

Work	Direct overlap	Boundary for ViewTree
VGGT [2]	Feed-forward multi-view geometry and camera estimation	Reconstruction backbone; no reasoning policy
Think3D [3]	Point-cloud memory, camera manipulation, iterative VLM reasoning, RL	Closest representation and interaction loop; primarily sequential
Chain-of-View [4]	Question-aligned anchors and iterative camera actions	Closest test-time active viewing pipeline; no confidence-guided branch fusion
AVIC [5]	Adaptive decision of whether/how much to imagine	Closest adaptive-compute motivation; uses a generative world model
cdViews [6]	Critical/diverse view selection and redundancy suppression	Closest multi-view selection; static candidate set
VisuoThink [7]	Multimodal tree search and branch selection	Closest search structure; not explicit 3D camera evidence on mobile
Mobile TTS [9]	Parallel test-time compute on smartphone NPUs	Closest mobile scaling theme; text reasoning rather than rendered spatial evidence

Novelty discipline: Do not claim point-cloud reconstruction, virtual-camera control, generic tree search, generic confidence pruning, or accuracy-minus-latency optimization independently. The defensible contribution is their reasoning-specific and device-aware integration: outcome-calibrated branch confidence, branch-local viewpoint hypotheses, disagreement-aware continuation, selective pose-tagged fusion, and cross-stage mobile execution.

1.7 Testable hypotheses

6. **H1 —** Accuracy: adaptive branching improves questions whose necessary evidence is distributed across viewpoints, compared with a single active-view trajectory.
7. **H2 —** Efficiency: confidence-guided pruning reaches the accuracy of fixed-width search with fewer rendered views, encoded visual tokens, and VLM decoding steps.
8. **H3 —** Fusion: selecting and jointly reasoning over retained branch evidence outperforms both choosing one winning branch and concatenating all generated views.
9. **H4 —** Calibration: a rollout-trained VLM value head ranks partial spatial trajectories more reliably than answer-token probability, verbal self-confidence, or random pruning.
10. **H5 —** Systems: prefix sharing, batched sibling processing, caching, and progressive rendering improve latency and energy without changing branch decisions.
11. **H6 —** Adaptation: a device-budget controller preserves a better accuracy–cost Pareto frontier across thermal states than a fixed branch width and depth.

2. Scope, assumptions, and success definition

2.1 In-scope task model

12. Input: a natural-language query, current camera view, and a recent buffer of historical RGB frames.
13. Environment: indoor or room-scale scenes that are static or slowly changing during the reasoning episode.
14. Action: a discrete or quantized SE(3) virtual-camera transformation over the reconstructed memory.
15. Observation: a rendering from the explicit memory, encoded into VLM-compatible visual tokens and tagged with its camera pose.
16. Output: a spatial answer, confidence estimate, and optional trace of view-control decisions.
17. Deployment: local inference on a smartphone-class SoC and an embedded platform such as Jetson Orin.

2.2 Explicit non-goals

18. A new 3D reconstruction architecture or photorealistic novel-view synthesis method.
19. Dynamic-scene tracking, full SLAM replacement, or long-term world-memory maintenance across major scene changes.
20. Low-level physical robot control, collision-free actuation, or manipulation planning.
21. Guaranteeing metric-grade geometry when the source video lacks parallax or coverage.
22. Exposing private chain-of-thought text as an evaluation artifact; controller tokens and externally verifiable actions are sufficient.

2.3 Success criteria

The method should be considered successful only if it improves the joint reasoning–systems frontier. A pure accuracy increase obtained by rendering many more views is insufficient, as is a latency reduction obtained by sacrificing the difficult multi-view questions that motivate branching. The primary result should be a Pareto curve of task accuracy against measured end-to-end latency and energy, accompanied by confidence calibration and failure analysis.

3. System architecture

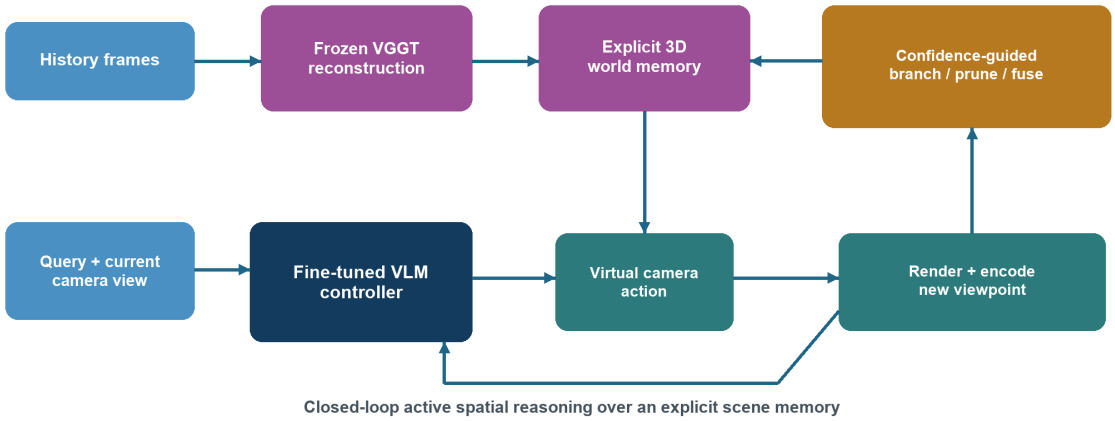

Figure 1. ViewTree architecture and closed-loop data flow.

3.1 Module boundaries

Module	Input	Output	Training status
Scene builder	Historical frames	Point cloud, camera poses, reliability metadata	Frozen
Renderer	World memory + camera pose	RGB/depth/visibility mask	Frozen
View encoder	Rendered view + pose	Visual state tokens	Frozen or lightly adapted
VLM controller	Query + branch context	MOVE/BRANCH/FUSE/STOP + answer	Fine-tuned
Confidence head	Branch hidden state	Probability of eventual correctness	Trained
Branch manager	Confidence values + runtime budget	Top-B retained branches	Deterministic
Mobile scheduler	Profile + live device state	Branch width/depth/fidelity budget	Runtime policy

3.2 State and action semantics

The world memory is not itself the VLM state. At step i, the reasoning state contains the query, current physical camera image, branch-local history of encoded rendered views, corresponding poses, controller

tokens, and remaining budget. A rendered observation is evidence acquired from the explicit memory; it is not a latent prediction of a learned dynamics model.

$$M = \Phi_{3D}(V) = \{P, C, \Pi\} \quad (1)$$

$$o_{i+1} = \mathrm{Render}(M, p_i \oplus a_i), \quad z_{i+1} = E(o_{i+1}, p_{i+1}) \quad (2)$$

$$h_{i+1} = f\theta(q, I\mathrm{current}, h_i, a_i, z_{i+1}, p_{i+1}, r_i) \quad (3)$$

The action a_i may be represented as a discrete camera primitive—forward, backward, left, right, up, down, yaw, and pitch—or as a quantized six-degree-of-freedom transformation. Discrete actions are preferred initially because they simplify demonstration generation, action validity checks, and mobile rendering caches.

3.3 Controller outputs

Control	Meaning	Typical condition
MOVE(a)	Acquire a new view on the current branch	Action is valid and budget remains
BRANCH	Fork shared context and try multiple camera actions	Several continuations may be useful
PRUNE	Remove low-confidence partial trajectories	Branch budget is exceeded or evidence is weak
FUSE	Jointly interpret retained pose-tagged evidence	Complementary evidence is available
STOP(y)	Return answer y and confidence	Further viewing has low expected value

4. Formal problem formulation

4.1 Resource-constrained belief-tree reasoning

ViewTree can be formulated as a partially observable decision process over a reconstructed scene. The physical scene is only partially represented by the world memory, and each virtual-camera action reveals a new projection. The controller maintains several branch-local belief states when one continuation is insufficient. The objective is to maximize final answer correctness while satisfying device limits.

$$\max\pi \; E[RQA] \quad \text{subject to} \quad E[L] \le L\max, \; E[E] \le E\max, \; E[M\mathrm{peak}] \le M\max, \; T\mathrm{device} \le T\max \quad (4)$$

L is end-to-end latency, E is energy, Mpeak is peak resident memory, and Tdevice is a thermal constraint or proxy. Unlike a single accuracy-minus-latency coefficient, explicit constraints give each target device an interpretable operating envelope.

4.2 Model-driven control instead of manual branching thresholds

Answer uncertainty, action ambiguity, and expected information gain remain useful explanations for why branching occurs, but they need not be implemented as three manually tuned thresholds. During training, the system evaluates alternative control decisions from the same state using continuation outcomes.

$$\hat{Q}(s_i, u) = (1/K)\, \Sigma_k\, R(\tau_{i,u}^{(k)}), \quad u \in \{\text{MOVE, BRANCH, FUSE, STOP}\} \quad (5)$$

$$u_i^* = \arg\max_u \hat{Q}(s_i, u) \quad (6)$$

This supplies demonstrations and preferences for the control policy. At inference time, the learned controller proposes the structure of the reasoning tree, while the device budget determines how many branches can be executed.

5. Method design

5.1 Query-conditioned initialization

The VLM first receives the query and current camera image. It identifies the spatial relation being requested, target entities, and likely missing evidence. This initial step does not require a separate object detector unless the implementation needs task-conditioned cropping. The controller emits either an answer, a camera action, or a BRANCH request.

5.2 World-memory construction and reuse

VGGT processes the recent frame buffer once to estimate camera parameters and scene geometry. The resulting point cloud is retained across all reasoning branches and, when appropriate, across multiple queries over the same recent environment. Reconstruction confidence or point density may be retained only as a rendering-validity signal; it should not become a hand-weighted semantic branch score. The first implementation should freeze VGGT to isolate the reasoning contribution.

5.3 Closed-loop view acquisition

For a MOVE decision, the branch manager validates the proposed camera transformation, renders the view, and invokes the visual encoder. The camera pose is embedded as a compact token sequence or

learned vector and appended next to the visual state. The VLM then updates only that branch's context. This observe–act–observe loop continues until the controller requests branching, fusion, or termination.

5.4 Branch creation and shared prefixes

When the controller emits BRANCH, the current reasoning prefix is copied conceptually but shared physically. Candidate camera actions are sampled from the policy's action distribution without replacement. Each child receives the same query and pre-branch observations but maintains its own post-branch visual tokens, controller outputs, and confidence. Independent continuation prevents evidence from one hypothesis from prematurely changing the reasoning of another.

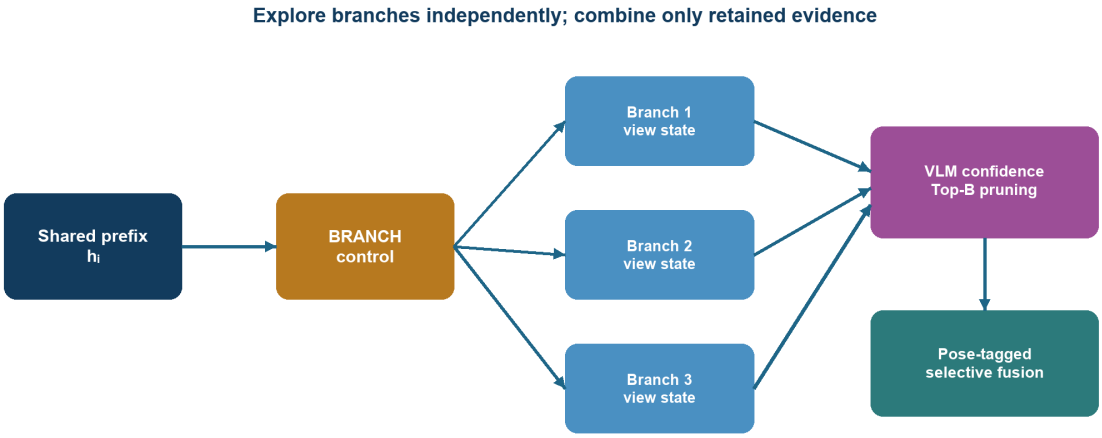

Figure 2. Branch-local exploration, confidence pruning, and selective fusion.

5.5 Confidence-guided branch pruning

A lightweight value head is attached to a designated confidence token in the VLM hidden state. For branch b, it predicts the probability that continuing from the current partial trajectory will eventually produce the correct final answer—not merely the probability of the branch's provisional answer token.

$$c\_b = V\varphi(h\_b) = \Pr(\text{final answer is correct} \mid \text{branch } b) \quad (7)$$

Given the runtime branch budget B_t, the pruning rule is deliberately simple:

$$Bkeep = \text{TopK}(\{c\_b : b \in B\}, B_t) \quad (8)$$

This eliminates manually weighted relevance, coverage, novelty, contradiction, and cost terms. Geometry is limited to two hard safeguards: reject a pose that cannot be rendered reliably, and collapse exact or near-identical cached views. The VLM confidence controls semantic pruning; the systems controller controls capacity.

5.6 Disagreement-aware continuation

Confidence alone must not convert the tree into a winner-take-all selector. If several retained branches produce the same answer with high confidence, ViewTree can fuse and stop. If one branch is clearly stronger, weaker branches can be discarded. If multiple high-confidence branches disagree, the disagreement is evidence that the current observations support competing spatial explanations. The controller should preserve those branches, acquire a disambiguating view, or invoke pairwise branch comparison.

$$P\varphi(b_1 \succ b_2 \mid q, o\_b_1, p\_b_1, o\_b_2, p\_b_2) \quad (9)$$

A pairwise comparison head is optional. The minimal implementation can retain the top-B branches and train the main controller to react to answer disagreement. Pairwise comparison should be added only if absolute confidence calibration proves insufficient.

5.7 Selective pose-tagged fusion

After pruning, the surviving branches are assembled as a structured multi-view input. Each view remains separated by a branch identifier and camera-pose token. The VLM attends jointly to the retained evidence and outputs either the final answer or another action. This avoids constructing a manually engineered geometric fusion score while still making coordinate changes explicit.

$$Xfuse = [q;\ \langle b_1,p_1\rangle\ ,o_1;\ \dots;\ \langle b\_m,p\_m\rangle\ ,o\_m] \quad (10)$$

$$\hat{y}, c\_fuse, u\_next = f\theta(Xfuse) \quad (11)$$

The model should be trained on complementary, redundant, and distracting branch combinations. Without such training, concatenating additional views may increase confidence while reducing correctness.

5.8 Stopping

STOP should be learned from outcome and cost, not triggered solely by a fixed confidence threshold. During training, trajectories that continue after the answer is already recoverable receive unnecessary resource cost, while trajectories that stop before acquiring required evidence fail the QA objective. At deployment, a hard deadline can force termination; otherwise, the learned control policy decides whether another view is worthwhile.

5.9 Inference algorithm

23. Construct or retrieve world memory M from the historical frame buffer.
24. Initialize a root branch with the query, current image, initial camera pose, and runtime budget.

25. For every active branch, let the VLM emit MOVE, BRANCH, FUSE, or STOP and a branch confidence value.
26. Execute requested camera actions, batch sibling rendering/encoding where possible, and append the observations only to their corresponding branches.
27. If active branches exceed B_t, keep the Top-B_t branches according to calibrated VLM confidence after validity and duplicate checks.
28. If high-confidence branches disagree, allocate a disambiguating step when resources permit; otherwise retain the best affordable hypotheses.
29. When FUSE is selected, jointly present the retained pose-tagged evidence to the VLM.
30. Return the answer when STOP is selected or the hard resource deadline is reached.

6. Training procedure

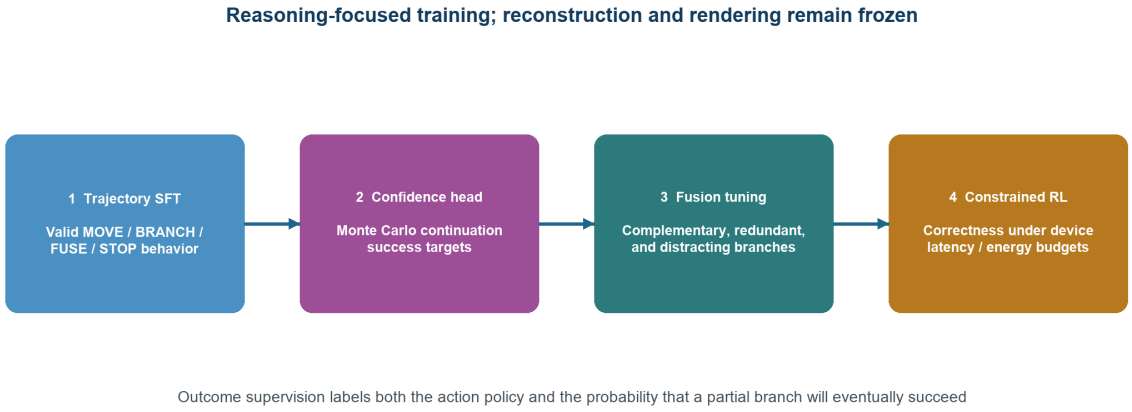

Figure 3. Four-stage ViewTree training pipeline.

6.1 Episode construction

Each training episode contains historical views V, a spatial query q, the ground-truth answer y*, and a world-memory instance M. If the source dataset supplies a scan, the same rendering interface should be used to produce training views; if it supplies only video, VGGT reconstructs M. Training and testing must be split by scene rather than by question to prevent the model from memorizing geometry.

$$e = (V, q, y^*, M, \text{device profile}) \quad (12)$$

6.2 Stage I — trajectory supervision

A strong teacher VLM or search procedure proposes camera actions, observes the executed renderings, and continues until it answers or exhausts a step budget. Successful trajectories teach valid action syntax and basic viewpoint exploration. To train control decisions, generate alternatives from the same state—continue one path, branch into several, fuse current evidence, or stop—and label the alternative with the highest rollout return.

$$L\_SFT = -\Sigma_i \log \pi\theta(u_i^* \mid s_i) - \log p\theta(y^* \mid s\_T) \quad (13)$$

The SFT corpus should balance easy questions that require no exploration, single-view-recovery questions, single-chain questions, and genuinely complementary multi-branch questions. Otherwise the policy may learn to branch unconditionally or never branch.

6.3 Stage II — outcome-calibrated confidence

For every collected partial branch state, sample K stochastic continuation rollouts using the current policy. A rollout receives one when its final answer is correct and zero otherwise. Their mean is a soft target for eventual branch success.

$$v\_b^* = (1/K)\, \Sigma_k\, 1[\hat{y}\_b^{(k)} = y^*] \quad (14)$$

$$L\_conf = -v\_b^* \log c\_b - (1-v\_b^*) \log(1-c\_b) \quad (15)$$

This label is preferable to immediate answer correctness because an intermediate branch may still be recoverable after another view. It also provides training examples from unsuccessful reasoning trees. The value head can share the VLM backbone while updating only the head and final transformer blocks initially.

6.4 Confidence calibration and auditing

After value-head training, fit one temperature on a held-out calibration split. Report Brier score, expected calibration error, negative log-likelihood, AUROC for successful versus failed branches, and Top-K branch recall. Calibration must be checked separately by task type, branch depth, reconstruction quality, and device budget because confidence may shift across these conditions.

$$c\_b\text{^}cal = \sigma(\ell\_b \,/\, T) \quad (16)$$

6.5 Stage III — complementary-evidence fusion

Construct three controlled branch sets: complementary sets in which the fused evidence is necessary, redundant sets that repeat the same information, and distractor sets containing irrelevant or unreliable

views. Train the VLM to answer from the structured pose-tagged input and to choose FUSE only when the combined evidence is sufficient.

$$L\_fuse = -\log p\theta(y^* \mid X\_fuse) \quad (17)$$

A particularly important training subset contains examples for which every individual branch is wrong or uncertain but the fused set is correct. These examples distinguish branch-and-fuse reasoning from simply selecting the most confident trajectory.

6.6 Stage IV — resource-constrained RL

Sample a group of complete reasoning trees for each question. Measure answer correctness and the actual or profiled device costs of each tree. Optimize the VLM policy using a GRPO-style group-relative objective. Device constraints enter through automatically updated dual variables rather than several manually tuned scalar penalties.

$$R\_g = R\_QA - \lambda\_L(L\_g - Lmax) - \lambda\_E(E\_g - Emax) - \lambda\_M(M\_g - Mmax) \quad (18)$$

$$A\_g = (R\_g - \text{mean}(R_1{:}G)) \,/\, (\text{std}(R_1{:}G) + \varepsilon) \quad (19)$$

$$\lambda\_L \leftarrow [\lambda\_L + \eta\lambda(L\_g - Lmax)]_+ \quad (20)$$

Use curriculum budgets during RL. Begin with shallow single-chain trajectories, introduce two-way branches after action validity stabilizes, then expose the controller to varying runtime budgets. Randomizing the budget during training prevents overfitting to one branch width and allows a single model to serve several operating points.

6.7 Joint objective and update schedule

$$L = L\_RL + L\_conf + L\_fuse \quad (21)$$

31. Freeze VGGT and the renderer throughout the main study.
32. Warm-start the VLM controller with SFT before any tree-level RL.
33. Train the confidence head from replayed partial states; refresh targets periodically as the policy changes.
34. Interleave fusion batches with policy rollouts so RL does not collapse into winner-take-all branch selection.
35. Apply stop-gradient to most of the VLM backbone during early confidence training if value updates destabilize camera control.
36. Maintain a held-out calibration set that is never used for policy updates.

6.8 Data quality controls

37. Reject demonstrations containing invalid camera poses, missing renderings, or answers copied from unavailable evidence.
38. Store the exact observations supplied to each branch so training labels can be independently replayed.
39. Balance positive and negative branch outcomes across depths to prevent confidence from becoming a proxy for branch length.
40. Prevent scene leakage by splitting at the environment/scan level before generating trajectories.
41. Generate counterfactual examples where additional views are redundant or harmful, not only helpful.
42. Audit teacher traces for action diversity; otherwise the student may inherit a narrow camera-motion prior.

7. Mobile systems design

7.1 End-to-end workload

Each additional branch can trigger four costs: camera-pose processing, point-cloud rendering, visual encoding, and VLM decoding. A mobile contribution must optimize the combined pipeline rather than count only camera actions. The evaluation should therefore report phase-level latency and energy, consistent with recent evidence that vision-language inference behaves differently across mobile CPU, GPU, and NPU phases [8].

7.2 Shared-prefix VLM execution

All children of a branch point share the same query and pre-branch multimodal context. ViewTree should retain the corresponding VLM KV cache once and allocate separate post-branch cache pages only after the fork. Copy-on-write cache management reduces memory growth from O(B·prefix) toward O(prefix + B·suffix). The implementation must verify whether the target runtime exposes cache-page sharing; otherwise it should emulate sharing through prefix replay batching and quantify the overhead.

7.3 Batched sibling rendering and encoding

Sibling camera poses are known simultaneously. Render them in one point-cloud pass when the graphics backend supports multi-view framebuffers, then batch the resulting images through the vision encoder. This converts several small, poorly utilized invocations into one larger batch and is especially important on NPUs whose efficiency depends on tensor shape and pipeline phase.

7.4 Progressive rendering fidelity

Candidate branches first receive a low-resolution or sparsely sampled preview. The confidence head ranks branches from the preview state; only retained branches are re-rendered or re-encoded at full fidelity. The study must test whether preview confidence preserves Top-K branch recall, because low-resolution filtering is useful only if it rarely removes the eventual successful trajectory.

7.5 Pose-indexed observation cache

Quantize camera poses into cache keys containing position, orientation, field of view, render resolution, and world-memory version. Cache both rendered images and encoded visual tokens. Reuse is expected when sibling branches converge, when the controller revisits a location, or when several questions inspect the same scene. The cache requires a bounded eviction policy and must be invalidated after world-memory updates.

7.6 Runtime budget controller

An offline device profile records the cost of reconstruction, rendering, visual encoding, VLM prefill, and decoding across batch sizes and resolutions. At runtime, the controller observes the deadline, available memory, battery/energy mode, thermal headroom, and processor contention. It selects the affordable branch cap B_t, maximum remaining depth, and rendering fidelity. The VLM determines which branches are semantically promising; the controller determines how many can execute.

$$B_t, D_t, \rho_t = \text{Scheduler}(\text{profile}, \text{deadline}, \text{memory}, \text{temperature}, \text{contention}) \quad (22)$$

7.7 Mobile implementation variants

Variant	Enabled mechanism	Primary measurement
Reference	Sequential render/encode/decode; no caches	Correctness baseline
PrefixShare	Shared pre-branch KV state	VLM memory and prefill
SiblingBatch	Batched render + visual encoder	Accelerator utilization
Preview	Low-fidelity rank, full-fidelity survivors	Avoid wasted visual processing
PoseCache	Reuse renderings and visual tokens	Repeated-view cost
Full runtime	All mechanisms + adaptive scheduler	End-to-end Pareto frontier

8. Experiment plan

8.1 Research questions

43. **RQ1:** Does explicit-memory active viewing improve spatial QA over the current frame, historical-frame prompting, and full-view prompting?
44. **RQ2:** When does branching improve accuracy over a single virtual-camera trajectory?
45. **RQ3:** Does confidence-guided Top-B pruning retain successful branches better than token probability, verbal self-confidence, random pruning, and fixed beam selection?
46. **RQ4:** Does selective fusion outperform choosing the best branch and concatenating every generated view?
47. **RQ5:** How robust are branch confidence and fusion to reconstruction errors, sparse geometry, occlusion, and large viewpoint changes?
48. **RQ6:** Which mobile optimizations reduce latency, energy, and memory, and do they preserve branch decisions?
49. **RQ7:** Can one policy adapt across deadlines and thermal states, or is per-device/per-budget fine-tuning required?
50. **RQ8:** What question characteristics predict that branching will help, remain marginal, or hurt?

8.2 Dataset portfolio

Dataset family	Role	Planned use
VSI-Bench / related video-spatial tasks	Egocentric video and multi-step spatial reasoning	Primary continuity with prior mobile paper; evaluate reconstruction from video
BLINK Multi-view / MindCube	Cross-view and perspective reasoning	Measure single-chain vs branching benefit
OpenEQA	Embodied QA over 3D environments	Direct comparison with Chain-of-View
ScanQA	Question answering over ScanNet scenes	Controlled scan-based rendering and 3D-QA comparison
SQA3D	Situated 3D question answering	Agent-relative pose and orientation reasoning
Real-device indoor set	Homes, offices, libraries, corridors	Latency/energy/thermal study and deployment realism

Before finalizing the benchmark suite, verify that each dataset's license permits reconstruction, novel-view rendering, and released trajectory annotations. Use scan-provided geometry only as an upper-

bound condition; the primary setting should reconstruct from the same historical observations available to the method.

8.3 Task taxonomy

51. No-exploration tasks: answerable from the current view; tests whether the controller avoids unnecessary computation.
52. Single-view recovery: one additional viewpoint is sufficient.
53. Sequential tasks: several actions along one coherent trajectory are required.
54. Complementary-view tasks: evidence from different directions must be combined.
55. Ambiguous-action tasks: several plausible next camera moves exist.
56. Adversarial-view tasks: additional renderings are redundant, incomplete, or misleading.
57. Long-horizon tasks: camera and object relations must be maintained across multiple coordinate changes.

8.4 Baselines

Baseline	Configuration	Question answered
Current-view VLM	Query + current image	No memory or active viewing
History-frame VLM	Uniform/key-frame video prompting	Tests raw context scaling
Full-view grid	Predetermined renderings concatenated	Tests maximum static context
cdViews-style	Critical/diverse static view selection	Closest selection baseline
Think3D-style chain	One iterative camera trajectory	Closest explicit-memory reasoning baseline
Chain-of-View	Anchor selection + iterative adjustment	Closest training-free active-view baseline
Fixed-width tree	Expand B branches at each depth	Tests unconditional branching
AVIC-style sample/select	Generate several complete trajectories; select one	Tests adaptive amount without fusion
VisuoThink-style search	Generic multimodal tree selection	Tests general tree-search transfer
Oracle budget/view	Ground-truth best view or best branch subset	Upper bound and headroom

8.5 Main accuracy experiments

58. Evaluate all methods with the same underlying VLM, visual resolution, answer decoder, reconstruction, and maximum test-time resource envelope.

59. Report overall accuracy and accuracy by task taxonomy, especially complementary-view and no-exploration subsets.
60. Plot accuracy against number of rendered views, encoded visual tokens, VLM decode tokens, latency, and energy.
61. Match ViewTree and fixed-width baselines at equal view count and equal measured latency to separate algorithmic from systems gains.
62. Include scan-based and reconstructed-memory conditions to quantify how much error comes from reasoning versus reconstruction.

8.6 Confidence evaluation

Metric	Definition	Why it matters
Branch success AUROC	Ranking successful vs failed continuations	Primary pruning quality
Top-B recall	Whether an eventual successful branch survives	Direct operational measure
Brier score	Squared calibration error	Probability quality
ECE	Gap between confidence and empirical accuracy	Calibration visualization
Selective accuracy	Accuracy after stopping above confidence levels	Safe early stopping
Regret vs oracle	Correctness lost by pruning the oracle branch	Failure attribution

8.7 Core ablations

Ablation	Change	Target conclusion
No branching	Single learned trajectory	Is branching needed?
Always branch	Fixed B and depth	Does adaptation save cost?
Random prune	Random Top-B	Is branch scoring meaningful?
Token confidence	Answer-token probability	Does rollout value improve ranking?
Verbal confidence	VLM self-reported score	Does trained calibration matter?
Best-branch only	No cross-branch fusion	Is complementarity real?
Fuse all	No pruning	Does irrelevant evidence hurt?
No pose tags	Views without camera pose	Does coordinate context matter?
No disagreement rule	Always prune to confidence order	Are conflicts prematurely removed?
No resource constraints	Correctness-only RL	Does policy overspend?

8.8 Mobile systems experiments

63. Candidate platforms: the same smartphone and Jetson-class device used in the prior SpatialMind study, plus a server GPU reference for algorithmic throughput.
64. Measure cold and warm execution separately; report reconstruction amortized per query and also as an upfront cost.
65. Record p50, p95, and p99 end-to-end latency; energy per query; peak RAM/accelerator memory; visual-token count; cache hit rate; and steady-state temperature.
66. Run sustained sequences long enough to expose thermal throttling and repeat tests under controlled ambient temperature and battery state.
67. Break latency into reconstruction, rendering, visual encoding, prefill, decoding, branch management, and fusion.
68. Compare equal-answer-quality operating points, not only maximum-throughput settings.

8.9 Robustness and stress tests

Stress factor	Manipulation	Measurement
Frame sparsity	Reduce historical frames / baseline	Reconstruction and confidence degradation
Pose noise	Perturb camera estimates	Sensitivity of pose-tagged fusion
Point dropout	Random/structured geometry removal	Invalid rendering and confidence failure
Occlusion	Increase hidden target fraction	Branching benefit by difficulty
Dynamic distractor	Move irrelevant objects	Static-memory limitations
Budget shock	Change deadline mid-query	Scheduler adaptation and graceful degradation
Thermal throttling	Sustained workload	Operating-point stability
Confidence shift	Unseen scene/task category	Calibration under distribution shift

8.10 Statistical protocol

69. Use scene-level splits and paired evaluation so every method answers the same questions from the same observations.
70. Report bootstrap 95% confidence intervals over scenes and paired significance tests for answer correctness.

71. Use at least three training seeds for learned policies and report mean, standard deviation, and the best validation-selected checkpoint policy.
72. For device measurements, warm up the runtime, repeat each configuration sufficiently, and report the number of runs and ambient conditions.
73. Predefine the primary metric and primary benchmark before inspecting the final test results.
74. Avoid selecting branch budgets independently on the test set; tune them on validation data or derive them from explicit device constraints.

8.11 Primary plots and tables for the paper

75. Accuracy–latency and accuracy–energy Pareto curves across branch budgets.
76. Accuracy by task taxonomy, highlighting complementary-view questions.
77. Branch confidence reliability diagram and Top-B recall curve.
78. Search-tree visualization for one success, one unnecessary-branch case, and one failure.
79. Phase-level latency/energy breakdown with and without each mobile optimization.
80. Ablation table separating branching, pruning, fusion, pose tags, and resource-aware training.
81. Failure taxonomy separating reconstruction, action, confidence, fusion, and systems-deadline errors.

9. Implementation roadmap

Phase	Goal	Main work	Exit criterion
M0	Reproduce explicit-memory chain	VGGT → renderer → encoder → VLM camera loop	Think3D-style baseline runs end to end
M1	Controller SFT	Action grammar and single-chain trajectories	Valid-action rate and baseline accuracy stabilize
M2	Tree runtime	Forked contexts, branch manager, Top-B execution	Fixed-width tree reproducible
M3	Confidence head	Continuation rollouts, calibration, pruning	Top-B recall exceeds simpler confidence baselines
M4	Fusion	Pose-tagged multi-branch training	Complementary-view subset improves
M5	Constrained RL	Joint branch/fuse/stop policy under budgets	Adaptive Pareto gain over fixed search
M6	Mobile runtime	Prefix sharing, batching, preview, cache, scheduler	Real-device latency/energy gains

Phase	Goal	Main work	Exit criterion
M7	Full evaluation	Baselines, ablations, robustness, statistics	Paper figures and artifact freeze

9.1 Minimum viable prototype

82. Frozen VGGT reconstruction from historical frames.
83. Point-cloud renderer supporting a small discrete camera-action set.
84. One VLM controller with MOVE, BRANCH, FUSE, and STOP tokens.
85. Two-way branching with shared pre-branch context.
86. One rollout-trained confidence head and Top-2 pruning.
87. Pose-tagged fusion of at most two retained branches.
88. Latency and memory instrumentation on one target device.

9.2 Recommended order of empirical decisions

89. Confirm that the basic explicit-memory loop improves over historical-frame prompting on the selected tasks.
90. Identify a substantial complementary-view subset; without it, branch fusion cannot become a central claim.
91. Test whether the rollout-trained confidence head retains the oracle-success branch at small B.
92. Validate that fusion improves over best-branch selection before investing in complex mobile scheduling.
93. Profile which phase dominates on the target device, then implement the systems mechanisms with the largest measured headroom.
94. Only after these checks, run constrained RL and large-scale evaluation.

10. Risks, failure modes, and mitigations

Risk	Failure	Mitigation
Confidence is miscalibrated	Correct branch is pruned	Rollout targets, held-out temperature calibration, Top-B recall monitoring
Branch collapse	Policy repeats similar camera actions	Action sampling without replacement; diverse teacher trajectories
Always-branch behavior	Accuracy rises but mobile cost explodes	Easy/no-exploration SFT examples and constrained RL

Risk	Failure	Mitigation
Winner-take-all behavior	Complementary evidence never fused	Fusion-specific training where no single branch succeeds
View overload	Fusion confidence rises while accuracy falls	Distractor/redundancy training and explicit FUSE control
Reconstruction holes	Rendered views contain artifacts	Hard render-validity guard and scan-vs-reconstruction analysis
Pose-token misuse	VLM ignores coordinate changes	Pose perturbation tests and action/pose auxiliary supervision if needed
RL reward hacking	Early stopping or answer-format shortcuts	Verifiable QA reward, trace replay, invalid-action penalties
Memory pressure	Branch caches exceed device capacity	Copy-on-write KV cache, bounded pose cache, budget-aware eviction
Weak novelty	Reviewers see Think3D + tree search	Center evidence complementarity, calibrated pruning, and measured mobile co-design

10.1 Go/no-go checks

95. Go if at least one benchmark contains enough questions where two complementary branches outperform the best single branch.
96. Go if VLM confidence ranks eventual branch success materially better than token confidence and random pruning.
97. Go if adaptive Top-B search matches fixed-width accuracy with substantially fewer views or lower measured cost.
98. Reframe as an active-view reasoning paper if mobile optimizations provide little gain beyond generic batching.
99. Reframe as a systems paper only if real-device co-design produces a clear Pareto improvement and remains stable under sustained load.
100. Stop or redesign if fusion rarely beats selecting the best trajectory; the central branch-and-fuse claim would then be unsupported.

11. Paper positioning

11.1 Recommended title

Title: ViewTree: Resource-Aware On-Device Spatial Reasoning via Confidence-Guided Viewpoint Branching and Fusion

11.2 One-sentence pitch

Pitch: ViewTree turns a reconstructed 3D scene into an adaptive reasoning tree in which a lightweight VLM explores viewpoint hypotheses independently, retains them by calibrated probability of eventual correctness, and fuses complementary evidence under a mobile-device budget.

11.3 Proposed contributions

101. A closed-loop spatial-reasoning controller that can continue, branch, fuse, or stop while manipulating an explicit 3D memory through virtual camera actions.
102. An outcome-calibrated VLM confidence head for pruning partial camera trajectories according to eventual answer correctness, with disagreement-aware preservation of competing hypotheses.
103. A pose-tagged selective-fusion mechanism trained specifically on complementary, redundant, and distracting viewpoint evidence.
104. A mobile runtime that shares branch prefixes, batches sibling visual processing, applies progressive rendering, caches pose-indexed states, and adapts the tree to live resource constraints.
105. A comprehensive evaluation separating reasoning gains, calibration quality, reconstruction effects, and real-device systems performance.

11.4 Claims to avoid

106. ‘The first VLM to manipulate a reconstructed 3D scene.’
107. ‘The first adaptive imagination system for spatial reasoning.’
108. ‘The first multimodal reasoning tree.’
109. ‘Geometry-aware pruning’ if the actual pruning decision is mainly VLM confidence.
110. ‘On-device’ without reporting real-device end-to-end latency, energy, memory, and sustained thermal behavior.
111. Numerical performance claims before the experiment suite and statistical protocol are complete.

11.5 Suggested introduction logic

112. Establish that spatial evidence is distributed across viewpoints and that small local VLMs cannot infer all hidden geometry from raw video.
113. Explain why generative imagination can be inconsistent and expensive, motivating explicit reconstructed memory.
114. Show that the entire 3D scene is too redundant and a single active-view trajectory is brittle.
115. Identify the unresolved systems question: when are multiple trajectories worth their cost?
116. Introduce confidence-guided branch-local exploration, selective fusion, and the mobile runtime as one integrated answer.
117. State contributions in terms of learned conditional computation and measured deployment, not reconstruction novelty.

12. Pre-submission evidence checklist

☐ A complementary-view subset is explicitly defined and independently validated.

☐ ViewTree beats the best single trajectory on that subset.

☐ Confidence Top-B recall and calibration are reported, not only final QA accuracy.

☐ Best-branch, fuse-all, and selective-fuse comparisons are included.

☐ Think3D, Chain-of-View, cdViews-style selection, and fixed-width search are represented fairly.

☐ All methods share the same VLM, reconstruction, view budget, and answer evaluation where possible.

☐ Scan geometry and reconstructed geometry results are separated.

☐ End-to-end mobile cost includes rendering, encoding, VLM prefill/decode, and control overhead.

☐ Latency distributions, energy, memory, cache behavior, and sustained temperature are measured.

☐ Resource constraints and operating points are selected without test-set tuning.

☐ At least three training seeds and scene-level confidence intervals are provided.

☐ Failure cases distinguish reconstruction, action, confidence, fusion, and deadline errors.

☐ The paper states clearly what is frozen, trained, profiled, and executed on device.

☐ The artifact records prompts, action spaces, rendering parameters, device profiles, and branch traces.

References and design anchors

[1] SpatialMind: Spatially Aware On-Device Embodied AI via Viewpoint Integration.

[2] Wang et al. VGGT: Visual Geometry Grounded Transformer. arXiv:2503.11651, 2025. arXiv abstract

[3] Zhang et al. Think3D: Thinking with Space for Spatial Reasoning. arXiv:2601.13029, 2026. arXiv abstract

[4] Zhao et al. CoV: Chain-of-View Prompting for Spatial Reasoning. arXiv:2601.05172, 2026. arXiv abstract

[5] Yu et al. When and How Much to Imagine: Adaptive Test-Time Scaling with World Models for Visual Spatial Reasoning. arXiv:2602.08236, 2026. arXiv abstract

[6] Wang et al. 3D Question Answering via only 2D Vision-Language Models (cdViews). arXiv:2505.22143, 2025. arXiv abstract

[7] Wang et al. VisuoThink: Empowering LVLM Reasoning with Multimodal Tree Search. arXiv:2504.09130, 2025. arXiv abstract

[8] Murthy et al. Phase Matters: Characterizing Heterogeneous Vision-Language Inference on a Mobile SoC. arXiv:2606.27906, 2026. arXiv abstract

[9] Hao et al. Scaling LLM Test-Time Compute with Mobile NPU on Smartphones. arXiv:2509.23324, 2025. arXiv abstract

[10] Guo et al. On Calibration of Modern Neural Networks. ICML, 2017. arXiv abstract

> **Document status:** This is a research design and experiment plan, not a record of completed results. Replace proposed platforms, dataset choices, and success criteria only after implementation constraints and dataset licenses are confirmed.